

We Need More Butterflies! Aumento de Dados com Modelos Generativos

João Fernandes 2021219036¹ e David Ferreira 2020248052²

Departamento de Engenharia Informática, Faculdade de Ciências e Tecnologias da
Universidade de Coimbra

Resumo Este relatório descreve a implementação de uma pipeline de classificação de borboletas com aumento de dados por modelos generativos. O dataset contém 5199 imagens RGB rotuladas, distribuídas por 75 classes, e 1300 imagens de teste para submissão Kaggle. A CNN baseline foi mantida sem alterações na arquitectura, no optimizador e na função de perda. Foram treinados dois modelos generativos, um Variational Autoencoder e uma DCGAN. O VAE teve melhores métricas generativas globais, com SSIM de 0.485654, FID de 300.957306 e Inception Score de 2.269178. Mesmo assim, as imagens geradas não foram usadas de forma cega. A geração final combinou VAE condicionado por imagens reais e DCGAN, sempre com filtro de confiança da CNN e limite de diversidade por classe. Foram aceites 120 imagens sintéticas, aumentando o treino de 3639 para 3759 amostras. A baseline original obteve 0.6167 de Accuracy e 0.6135 de F1 ponderado no teste interno. A CNN treinada com dados aumentados obteve 0.5808 de Accuracy e 0.5737 de F1 ponderado. Assim, o aumento sintético não melhorou o classificador final. A submissão Kaggle foi gerada com a baseline original, por ser o melhor modelo interno.

Keywords: Classificação de imagem · Modelos generativos · VAE · DCGAN · Aumento de dados

1 Introdução

O objectivo do trabalho é avaliar se modelos generativos conseguem melhorar a classificação de espécies de borboletas através de aumento de dados. O classificador de base é uma CNN definida no notebook e deve ser comparado com a mesma CNN treinada depois de adicionar imagens sintéticas ao conjunto de treino.

A restrição principal é manter a CNN baseline sem alteração estrutural. A arquitectura, o optimizador **Adam** e a **CrossEntropyLoss** são os mesmos na baseline original e no re-treino com dados aumentados. Esta opção torna a comparação directa: a diferença entre os resultados deve vir dos dados usados no treino, não de alterações no modelo.

A solução implementada tem cinco fases. Primeiro, o dataset é carregado, pré-processado e dividido em treino, validação e teste. Segundo, a CNN baseline

é treinada e avaliada. Terceiro, são treinados um VAE e uma DCGAN. Quarto, são calculadas métricas generativas, SSIM, FID e Inception Score. Quinto, são geradas imagens sintéticas e a CNN é treinada de novo com o conjunto aumentado. No fim, o código escolhe para submissão o classificador com melhor F1 ponderado no teste interno.

2 Problema e Dados

O problema é uma classificação multi-classe. Para cada imagem x , o modelo deve prever uma classe y entre 75 espécies de borboletas. A dificuldade vem da semelhança visual entre algumas espécies, da variação de pose, da escala da borboleta na imagem, do fundo e das diferenças de iluminação.

O ficheiro `train.csv` contém o nome do ficheiro e a etiqueta textual de cada imagem rotulada. A pasta `train` contém as imagens rotuladas. A pasta `test` contém as imagens usadas para a submissão Kaggle.

Tabela 1. Resumo dos dados usados na experiência.

Elemento	Valor
Imagens rotuladas	5199
Classes	75
Imagens no teste Kaggle	1300
Treino interno	3639
Validação interna	780
Teste interno	780

A divisão interna foi estratificada, com `random_state=42`. O conjunto rotulado foi dividido em 70% para treino, 15% para validação e 15% para teste. A estratificação é importante porque o problema tem 75 classes e cada classe precisa de continuar representada nos três subconjuntos.

3 Abordagem Implementada

3.1 Leitura e Pré-processamento

A classe `ButterflyDataset` encapsula a leitura dos dados e evita repetir lógica de acesso às imagens em várias fases do notebook. No construtor, o `DataFrame` é reindexado para garantir que os índices são contínuos, a pasta das imagens é guardada e as classes são ordenadas alfabeticamente. Esta ordenação fixa o mapeamento entre etiqueta textual e índice inteiro, porque sem isto as duas execuções poderiam usar ordens diferentes de classes e produzir previsões incompatíveis com a submissão.

No método `__getitem__`, cada imagem é aberta com PIL e convertida para RGB. A conversão para RGB evita inconsistências caso alguma imagem tenha

outro formato interno. O `Resize((64,64))` força todas as imagens a terem a mesma resolução, requisito necessário para formar batches e alimentar uma CNN com tensores de dimensão fixa. A função `ToTensor()` converte a imagem para tensor PyTorch no formato canais, altura e largura, e escala os pixels para o intervalo $[0, 1]$. Esta escala é compatível com a saída `sigmoid` dos modelos generativos usados mais tarde.

Os `DataLoader` usam `batch_size=32`. Este valor é dá-nos um gradiente e um consumo de memória estáveis. O treino usa `shuffle=True` para apresentar as imagens por ordem aleatória em cada epoch, para reduzir o risco de o modelo aprender padrões da ordem dos dados. A validação e o teste usam `shuffle=False`, porque nestas fases não há actualização de pesos e a ordem dos exemplos não deve influenciar as métricas.

3.2 CNN Baseline

A `BaselineCNN` tem três blocos convolucionais. Cada bloco contém duas convoluções 3×3 , `BatchNorm2d`, `ReLU` e `MaxPool2d`, que aprendem filtros locais, como contornos, texturas e padrões de asa. O kernel 3×3 é uma escolha comum porque capta contexto local com poucos parâmetros e permite executar um stack de várias camadas para construir padrões mais complexos.

A função `BatchNorm2d` normaliza as activaões por canal dentro de cada batch, ou seja, reduz variações fortes na distribuição das activaões ao longo do treino. Isto estabiliza a optimização, permite gradientes mais controlados e torna a rede menos sensível à escala das activaões produzidas pelas convoluções, usamos depois das convoluções para estabilizar a aprendizagem dos filtros.

A função `ReLU` aplica $\max(0, x)$, anula valores negativos e os valores positivos passam sem alteração. Esta função introduz a não linearidade, se não tivesse funções de activação, as camadas convolucionais stacked comportar-se-iam como uma transformação essencialmente linear, sem capacidade para representar relações visuais complexas. A `ReLU` também é computacionalmente simples e reduz o problema dos gradientes muito pequenos em activaões positivas.

A função `MaxPool2d` reduz a resolução espacial dos mapas de activação, ao manter o valor máximo em cada janela, diminui o custo computacional, aumenta o campo receptivo das camadas seguintes e torna as características menos sensíveis a pequenas translações. Este efeito é útil em imagens de borboletas, porque a posição exacta da asa ou do corpo pode variar entre imagens da mesma espécie.

Os canais evoluem de 3 para 64, depois para 128 e depois para 256, aumenta a capacidade de representação à medida que a resolução espacial diminui. As primeiras camadas detectam padrões simples e as camadas mais profundas combinam esses padrões em descritores mais específicos de cada espécie. No fim da parte convolucional, a `AdaptiveAvgPool2d((1,1))` transforma cada mapa de activação num único valor, esta camada resume a informação espacial e torna a entrada do classificador mais densa independente da grelha espacial final.

O classificador denso usa camadas lineares com `ReLU` e `Dropout(0.5)`. O `Dropout` desativa aleatoriamente metade das unidades durante o treino, ou seja, força a rede a não depender demasiado de um pequeno conjunto de neurónios,

funciona como uma regularização, e a saída final tem 75 unidades, uma por classe. A função `forward` extrai características, aplica `flatten` e devolve `logits`, são valores reais não normalizados.

A loss usada é `CrossEntropyLoss`, adequada para classificação multi-classe com classes mutuamente únicas. Em PyTorch, esta loss recebe directamente os `logits`. Internamente, combina `LogSoftmax`, que transforma os `logits` em log-probabilities normalizadas, com `NLLLoss`, que penaliza o modelo quando a log-probability da classe correcta é baixa. Esta combinação é numericamente estável, de maneira a que, o PyTorch evita calcular primeiro `softmax` e depois `log` de forma separada, o que poderia causar *overflow* ou *underflow* quando os `logits` são muito grandes ou muito pequenos.

O treino usa `Adam(lr=0.001)` durante um máximo de 50 epochs, ele adapta a taxa efectiva de actualização de cada parâmetro usando estimativas dos momentos dos gradientes. Isto é útil porque uma CNN tem muitos parâmetros e gradientes com escalas diferentes. Usa-mos o valor `lr=0.001` por ser uma taxa de aprendizagem padrão para o `Adam`: e suficientemente alta para permitir convergência em poucas dezenas de epochs, mas não tão alta que torne a otimização instável. O early stopping monitoriza a loss validation com paciência de 5, ou seja, evita parar por causa de uma única epoch pior, mas ao mesmo tempo, interrompe o treino se a validação deixar de melhorar de forma consistente e seguida, basicamente sempre que a loss validation melhora, os pesos são guardados em memória com `copy.deepcopy(model.state_dict())`. No fim, esses melhores pesos são carregados antes da evaluation no teste interno.

3.3 Variational Autoencoder

O VAE foi usado para aprender uma representação latente contínua das imagens. A ideia é comprimir cada imagem num vector latente e depois reconstruí-la, se o espaço latente aprender uma estrutura útil, pequenas alterações nesse vector podem produzir variações visuais da imagem original.

O encoder convolucional reduz a imagem $3 \times 64 \times 64$ até um tensor $256 \times 4 \times 4$. Este tensor é achatado e projectado para dois vectores de dimensão 128: `mu` e `logvar`. O vector `mu` representa o centro da distribuição latente de uma imagem e o vector `logvar` representa a variância em escala logarítmica, usa-se `logvar` porque é mais estável do que prever directamente a variância, pois evita valores negativos e facilita a optimização.

A dimensão latente 128 foi escolhida como compromisso, um espaço latente muito pequeno comprime demasiado a imagem e perde detalhes importantes das asas e um espaço muito grande pode aproximar-se de uma cópia directa da imagem e reduzir a capacidade de gerar variações suaves. Com 128 dimensões, o VAE tem capacidade suficiente para codificar padrões visuais, mas continua obrigado a resumir a imagem.

A função de reparametrização calcula $z = \mu + \epsilon\sigma$. O ruído ϵ vem de uma distribuição normal, isto permite criar uma amostra do espaço latente durante o treino sem bloquear a retropropagação, não tendo reparametrização, a ope-

ração de amostragem partia o fluxo de gradientes e o encoder não aprenderia correctamente.

O decoder começa com uma camada linear que reconstrói o tensor $256 \times 4 \times 4$ e usa convoluções transpostas até recuperar a resolução original. A activação final é `sigmoid`, porque o pré-processamento coloca os pixéis no intervalo $[0, 1]$.

A loss implementada é uma beta-VAE:

$$\mathcal{L}_{VAE} = \text{MSE}(\hat{x}, x) + \beta \cdot D_{KL}(q(z|x)||p(z)). \quad (1)$$

No código, `BETA_KL=0.001`, este valor reduz o peso da regularização KL e dá mais importância à reconstrução escolhemos este valor porque se a componente KL for demasiado forte, o VAE pode produzir imagens demasiado suavizadas e pouco úteis para a CNN e com `BETA_KL=0.001`, o treino preserva melhor informação visual, embora o espaço latente fique menos regular do que num VAE clássico com $\beta = 1$.

O VAE também usa `Adam(lr=0.001)`, aqui a taxa de aprendizagem é igual à da CNN porque o treino é supervisionado por uma reconstruction loss relativamente estável e a paciência do early stopping é 10, maior do que na CNN, pois a reconstruction loss do VAE pode estabilizar lentamente e pequenas melhorias podem aparecer mesmo depois de várias epochs a piorar.

3.4 DCGAN

A DCGAN foi usada como modelo generativo adversarial, ao contrário do VAE, não reconstrói imagens de entrada. O gerador aprende a transformar ruído aleatório em imagens e discriminador aprende a distinguir imagens reais de imagens geradas. O treino é competitivo, isto é, o discriminador melhora a detectar falsificações e o gerador melhora a produzir imagens que o discriminador aceita como reais.

O gerador recebe ruído de dimensão 100 no formato $100 \times 1 \times 1$, esta dimensão é suficiente para introduzir variedade e continuar compacta. Se o vetor fosse demasiado pequeno, o gerador teria pouca capacidade para variar as amostras e se fosse demasiado grande, o treino adversarial podia tornar-se ainda mais instável. As camadas `ConvTranspose2d` aumentam progressivamente a resolução até gerar imagens RGB 64×64 e a ativação final é `sigmoid`, para produzir valores no intervalo $[0, 1]$, compatíveis com a CNN e com o pré-processamento das imagens reais.

O discriminador recebe imagens $3 \times 64 \times 64$ e usa camadas convolucionais com `LeakyReLU`, esta função é usada em vez de `ReLU` porque mantém uma pequena inclinação para valores negativos, vai reduzir o risco de neurónios ficarem sem gradiente durante o treino adversarial. A saída é uma probabilidade, obtida por `sigmoid`, que indica se a imagem parece real.

A loss usada na DCGAN é `BCELoss`, é adequada porque o discriminador resolve uma tarefa binária (real ou gerado), onde as imagens reais recebem alvo 1 e as imagens geradas recebem alvo 0. O gerador é treinado com o objectivo inverso, quer que o discriminador atribua 1 às imagens falsas. O otimizador é

o `Adam(lr=0.0002, betas=(0.5,0.999))` e a taxa 0.0002 é mais baixa do que na CNN e no VAE porque as GANs são mais instáveis. Actualizações demasiado grandes podem fazer o gerador e o discriminador oscilar sem convergir, daí o parâmetro `beta1=0.5` que reduz a inércia do primeiro momento do Adam, é uma prática comum em DCGAN, para reagir melhor às mudanças rápidas do treino adversarial e o `beta2=0.999` mantém uma estimativa estável da variância dos gradientes.

3.5 Geração Sintética e Filtros

A geração final não usa apenas o modelo com melhor métrica generativa, o código avalia a utilidade das imagens para o classificador. Foram usados dois caminhos.

No VAE condicionado, cada imagem real de treino é primeiro codificada para μ , sendo depois aplicada uma pequena perturbação ao vector latente através de `VAE_LATENT_NOISE_STD=0.15`; em seguida, o decoder usa esse vector perturbado para gerar uma variante da imagem original. Este desvio padrão controla directamente a intensidade da alteração feita no espaço latente, pelo que um valor demasiado baixo tenderia a produzir quase reconstruções da imagem inicial, com pouco valor como data augmentation, enquanto um valor demasiado alto poderia afastar demasiado a amostra gerada da identidade visual da classe. Assim, o valor 0.15 foi escolhido como uma perturbação moderada, suficientemente pequena para manter a imagem próxima do exemplo real, mas ainda capaz de introduzir alguma variação visual.

Como o VAE condicionado parte de uma imagem real, existe um rótulo esperado para cada imagem gerada, por isso, a amostra só é aceite quando a CNN baseline concorda com esse rótulo e apresenta confiança de pelo menos 0.85. Esta regra é propositadamente exigente, uma vez que uma discordância da CNN pode indicar que a imagem perdeu características discriminativas importantes da classe original, tornando-a arriscada para treino supervisionado. Na DCGAN, pelo contrário, a imagem é gerada apenas a partir de noise e não existe rótulo real associado, pelo que a CNN baseline atribui um pseudo-label à amostra; neste caso, a imagem só é aceite se a maior probabilidade da `softmax` for pelo menos `CONFIDENCE_THRESHOLD=0.85`, reduzindo o risco de adicionar exemplos com pseudo-labels incertas. O custo desta filtragem é aceitar menos imagens, mas essa opção é preferível a introduzir muitas amostras com rótulos pouco fiáveis, que poderiam degradar o treino do classificador.

O parâmetro `MAX_SYNTHETIC_PER_CLASS=50` limita o número de imagens sintéticas aceites por classe, funcionando como um filtro de diversidade para impedir que uma classe dominante ocupe uma parte demasiado grande do dataset aumentado. Sem este limite, a DCGAN poderia reforçar sobretudo as classes que a CNN já reconhece com maior confiança, aumentando o enviesamento do treino e reduzindo ainda mais a cobertura das 75 classes. No resultado final, este limite teve impacto directo, uma vez que POPINJAY e RED CRACKER atingiram ambas as 50 imagens sintéticas aceites, mostrando que o filtro impediu uma concentração ainda maior nessas classes, embora não tenha sido suficiente para resolver a falta de diversidade global das imagens geradas.

4 Configuração Experimental

A Tabela 2 resume os parâmetros necessários para replicar a experiência.

Tabela 2. Parâmetros experimentais principais.

Componente	Configuração
Pré-processamento	<code>Resize((64,64)), ToTensor()</code> , imagens RGB
Divisão dos dados	70% treino, 15% validação, 15% teste, estratificada, <code>random_state=42</code>
Seed	<code>SEED=42</code> , usada em Python, NumPy, PyTorch e CUDA
Batch size	32
CNN baseline	3 blocos convolucionais, <code>BatchNorm2d</code> , <code>ReLU</code> , <code>MaxPool2d</code> , <code>AdaptiveAvgPool2d</code> , <code>Dropout(0.5)</code>
Classificador	50 epochs, <code>Adam(lr=0.001)</code> , <code>CrossEntropyLoss</code> , early stopping com paciência 5
VAE	Latente 128, <code>Adam(lr=0.001)</code> , <code>BETA_KL=0.001</code> , 50 epochs, early stopping com paciência 10
DCGAN	Ruído 100, <code>BCELoss</code> , <code>Adam(lr=0.0002, betas=(0.5,0.999))</code> , 50 epochs, early stopping com paciência 10
Métricas generativas	SSIM, FID e Inception Score em 128 imagens, batch interno 32, cálculo em CPU
Geração sintética	3639 candidatas VAE condicionadas, <code>NUM_IMAGES_TO_GENERATE=10000</code> , <code>BATCH_Z=100</code> , aceitação com confiança mínima 0.85
Limite por classe	<code>MAX_SYNTHETIC_PER_CLASS=50</code>
Dataset aumentado	3639 imagens reais e 120 imagens sintéticas, total 3759

O `SEED=42` foi aplicado a Python, NumPy, PyTorch e CUDA para tornar a experiência tão reprodutível quanto possível, uma vez que a seed influencia a inicialização dos pesos, a divisão dos dados e a ordem aleatória dos batches. Mesmo assim, em GPU pode continuar a existir alguma variação causada por operações não determinísticas, mas fixar a seed reduz bastante essa variabilidade, ou seja, permite comparar a baseline com o modelo aumentado em condições mais próximas e com menos ruído experimental.

As 50 epochs funcionam como limite máximo de treino, não como obrigação de treinar sempre até ao fim, porque o early stopping decide a paragem com base na evolução da validação. Esta separação é importante, já que o limite máximo dá tempo ao modelo para aprender, enquanto a paciência impede que o treino continue quando a loss de validação deixa de melhorar. No classificador foi usada paciência 5 porque a métrica supervisionada tende a reagir de forma mais directa, enquanto no VAE e na DCGAN foi usada paciência 10, dando mais margem a modelos generativos, onde as losses são normalmente menos regulares e podem oscilar mais antes de estabilizar.

Na geração sintética, `NUM_IMAGES_TO_GENERATE=10000` foi usado para criar muitas candidatas DCGAN antes da aplicação do filtro de confiança, porque esse filtro é exigente e gerar poucas imagens poderia resultar em poucas ou nenhuma amostras aceites para o dataset aumentado. O parâmetro `BATCH_Z=100` define quantos vectores de noise são processados de cada vez, ou seja, permite gerar as imagens de forma eficiente sem colocar as 10000 candidatas em memória ao mesmo tempo.

Para o classificador foram usadas Accuracy e F1 ponderado, porque estas métricas avaliam aspectos complementares do desempenho. A Accuracy mede a fracção total de previsões correctas e é uma métrica intuitiva, mas pode esconder maus resultados em classes com menos exemplos. O F1 combina precision e recall, isto é, tem em conta tanto os falsos positivos como os falsos negativos, e a versão ponderada calcula o F1 por classe e combina esses valores de acordo com o número de exemplos de cada classe. Como o problema tem 75 classes, esta métrica dá uma visão mais robusta do comportamento global do classificador, sendo por isso usada como critério principal na escolha do modelo para submissão, ficando a Accuracy apenas como desempate.

Para os modelos generativos foram usadas SSIM, FID e Inception Score, porque cada uma destas métricas observa a qualidade das imagens de uma perspectiva diferente. O SSIM mede semelhança estrutural entre imagens, valorizando padrões de luminância, contraste e estrutura, sendo útil para perceber se as imagens geradas preservam formas semelhantes às reais. O FID compara distribuições de características extraídas por uma rede pré-treinada, ou seja, quanto menor for o FID, mais próxima está a distribuição gerada da distribuição real nesse espaço de características. O Inception Score avalia se as imagens geradas são classificadas com confiança e se existe diversidade entre classes previstas, pelo que valores maiores são melhores. Estas métricas são úteis para avaliar qualidade global, mas não garantem que uma imagem sintética tenha o rótulo certo nem que melhore o treino da CNN, por isso foram complementadas com filtros downstream baseados na confiança do classificador.

5 Resultados

5.1 CNN Baseline

A baseline original ativou early stopping na epoch 49. A melhor validação ocorreu antes da paragem, com validation accuracy 0.6410 e F1 ponderado 0.6272 na epoch 44. No conjunto de teste interno, a baseline obteve 0.6167 de accuracy e 0.6135 de F1 ponderado.

Tabela 3. Resultado da baseline original no teste interno.

Modelo	Accuracy	F1 ponderado
Baseline CNN	0.6167	0.6135

5.2 Modelos Generativos

O VAE activou early stopping na epoch 33. A validação estabilizou perto de 190 de loss, depois de uma descida progressiva ao longo das primeiras epochs. A DCGAN activou early stopping na epoch 13 e o treino adversarial foi mais instável, com variações fortes na loss do gerador.

Tabela 4. Métricas dos modelos generativos.

Modelo	SSIM	FID	Inception Score	Desvio
VAE	0.485654	300.957306	2.269178	0.362002
DCGAN	0.053426	361.300812	1.908932	0.278091

O VAE ficou em primeiro lugar nas três métricas: maior SSIM, menor FID e maior Inception Score. Isto indica melhor qualidade visual global e maior proximidade à distribuição real, dentro da amostra usada para cálculo das métricas. No entanto, esta conclusão não basta para decidir o aumento de dados, a métrica generativa mede qualidade global e o treino supervisionado precisa de imagens que a CNN consiga associar a classes úteis.

5.3 Utilidade Downstream das Imagens Geradas

O diagnóstico de confiança confirma a diferença entre qualidade global e utilidade para classificação, uma vez que as métricas generativas indicam apenas se as imagens parecem plausíveis no conjunto, enquanto a CNN mostra se essas imagens são suficientemente reconhecíveis para serem usadas como exemplos de treino. Em 2000 amostras livres do VAE, a confiança média da CNN foi 0.585033 e a confiança máxima foi 0.71101, ou seja, nenhuma imagem atingiu o limiar 0.85, o que sugere que, apesar dos melhores resultados globais do VAE nas métricas generativas, as suas amostras livres não ficaram suficientemente próximas das regiões que a CNN associa a classes concretas. Já em 2000 amostras da DCGAN, a confiança média foi 0.697276, a confiança máxima foi 1.0 e 820 imagens passaram o limiar 0.85, mas essas imagens cobriram apenas 3 classes, isto é, a DCGAN gerou amostras que a CNN reconheceu com mais confiança, embora com pouca diversidade útil para um problema com 75 classes.

O diagnóstico do VAE condicionado também mostrou limitações claras, porque, em 1000 reconstruções com $z = \mu$, houve apenas 60 concordâncias com o rótulo real e só 7 imagens tiveram simultaneamente concordância e confiança superior a 0.85. Quando foi aplicada perturbação latente com desvio padrão 0.15, o resultado manteve-se praticamente igual, com 61 concordâncias e novamente apenas 7 imagens com concordância e confiança superior a 0.85, o que indica que a perturbação introduziu alguma variação, mas não melhorou de forma relevante a capacidade do VAE gerar amostras reconhecíveis e úteis para reforçar o treino supervisionado.

Tabela 5. Resumo da geração sintética aceite.

Fonte	Candidatas	Aceites
VAE condicionado	3639	19
DCGAN	10000	101
Total	13639	120

As 120 imagens aceites cobriram 5 das 75 classes. A distribuição foi muito concentrada: POPINJAY teve 50 imagens, RED CRACKER teve 50, BANDED PEACOCK teve 17, ADONIS teve 2 e RED POSTMAN teve 1. O limite por classe impediu que as duas classes dominantes ultrapassassem 50 exemplos cada, mas não resolveu a falta de cobertura global.

5.4 Re-treino com Dataset Aumentado

O dataset aumentado juntou 3639 imagens reais e 120 imagens sintéticas, para um total de 3759 amostras. A CNN foi treinada de novo do zero, com a mesma arquitectura, o mesmo optimizador e a mesma loss da baseline original. No treino aumentado, o ciclo chegou à epoch 50. A validação final teve Accuracy 0.6269 e F1 ponderado 0.6051.

No teste interno, o modelo aumentado teve pior desempenho do que a baseline original.

Tabela 6. Comparação final no conjunto de teste interno.

Modelo	Treino real	Treino sintético	Accuracy	F1 ponderado
Baseline original	3639	0	0.6167	0.6135
Baseline com dados gerados	3639	120	0.5808	0.5737
Diferença	0	120	-0.0359	-0.0398

O aumento reduziu a Accuracy em 3.59 pontos percentuais e o F1 ponderado em 3.98 pontos percentuais. Este resultado é coerente com a baixa diversidade das imagens aceites. O aumento reforçou poucas classes e não melhorou a representação global do problema.

5.5 Submissão e Gravação dos Modelos

O notebook guarda quatro ficheiros na pasta `/content/drive/MyDrive/content/ACA/models`, nomeadamente `baseline_cnn.pth`, `vae_model.pth`, `gan_generator.pth` e `augmented_model.pth`, que correspondem aos modelos treinados ao longo da experiência. Estes ficheiros guardam os respectivos `state_dict`, isto é, os pesos e parâmetros aprendidos por cada modelo, em vez de guardarem a definição completa das classes PyTorch. Esta opção é adequada neste projecto, porque as arquitecturas já estão definidas no notebook e, para voltar a avaliar os modelos ou gerar a submissão, basta reconstruir a mesma arquitectura e carregar os valores aprendidos.

A célula de submissão Kaggle compara os classificadores treinados e escolhe aquele que teve melhor desempenho interno, usando o F1 ponderado como critério principal e a Accuracy apenas como desempate. Esta regra torna a escolha mais robusta, porque evita seleccionar um modelo apenas por acertar mais exemplos no total quando esse ganho pode estar concentrado em poucas classes, deixando pior o equilíbrio global entre as 75 espécies. Como a baseline original obteve 0.6135 de F1 ponderado, enquanto o modelo aumentado ficou em 0.5737, a variável `submission_model` ficou definida como a baseline original, e foi esse modelo que gerou o ficheiro `submission.csv`, guardado em `/content/drive/MyDrive/content/ACA/submission.csv` com 1300 previsões.

6 Discussão Técnica

O ponto técnico principal é que o melhor modelo generativo segundo SSIM, FID e Inception Score não produziu, por si só, o melhor data augmentation, porque estas métricas avaliam a qualidade global das imagens, mas não confirmam se cada amostra gerada é útil para treinar um classificador supervisionado. O VAE teve melhores métricas globais, no entanto as suas imagens livres não passaram o filtro de confiança da CNN, o que indica que, apesar de serem globalmente mais próximas da distribuição real, ficaram demasiado afastadas das regiões que a CNN reconhece como classes úteis.

A DCGAN teve o comportamento oposto, gerando imagens que a CNN classificou com maior confiança, mas concentradas em poucas classes, o que é compatível com colapso parcial de modos. Isto significa que o generator aprendeu padrões visuais que a CNN associa fortemente a algumas espécies, mas não conseguiu cobrir de forma equilibrada a distribuição completa das 75 classes. Assim, a DCGAN produziu amostras aparentemente mais úteis para a CNN, mas com diversidade insuficiente para melhorar o treino global.

O filtro `CONFIDENCE_THRESHOLD=0.85` reduziu pseudo-labels fracas, mas também favoreceu classes que a baseline já reconhecia com maior segurança, ou seja, classes mais fáceis passaram mais vezes pelo filtro. O limite `MAX_SYNTHETIC_PER_CLASS=50` ajudou a reduzir a concentração extrema, impedindo que uma só classe acumulasse demasiadas imagens sintéticas, mas não conseguiu criar diversidade onde o generator não a produziu. Por isso, a pipeline acabou por aceitar poucas imagens e essas imagens ficaram distribuídas de forma pouco equilibrada.

A diferença entre validação e teste também é relevante, porque o modelo aumentado terminou com validação final razoável, mas perdeu desempenho no teste interno, sugerindo que as imagens sintéticas não acrescentaram variação robusta ao treino. Pelo contrário, podem ter introduzido ruído ou reforçado padrões demasiado específicos de poucas classes, o que prejudicou a capacidade de generalização da CNN para exemplos reais não vistos.

O resultado final não invalida a abordagem generativa, mas mostra que, neste problema, a geração não condicionada por classe é insuficiente para melhorar a classificação. Um passo seguinte mais directo seria usar uma Conditional GAN ou um VAE condicionado por classe, porque isso permitiria gerar amostras por

espécie e controlar melhor a cobertura do dataset aumentado. Outra melhoria simples seria guardar as imagens aceites em disco, permitindo auditoria visual, comparação entre execuções e replicação exacta do conjunto aumentado.

7 Conclusão

Foi implementada uma pipeline completa para classificação de borboletas com data augmentation generativo, começando na preparação dos dados, passando pelo treino da CNN baseline, do VAE e da DCGAN, e terminando com a avaliação por SSIM, FID e Inception Score, a filtragem downstream por confiança da CNN, a criação do dataset aumentado, o re-treino da CNN e a geração da submissão Kaggle.

A baseline original foi o melhor classificador final, com 0.6167 de Accuracy e 0.6135 de F1 ponderado, enquanto o modelo treinado com 120 imagens sintéticas obteve 0.5808 de Accuracy e 0.5737 de F1 ponderado. Assim, nesta execução, o aumento sintético não melhorou a generalização.

A análise técnica explica este resultado, uma vez que o VAE teve melhores métricas generativas globais, mas quase não produziu imagens úteis para a CNN, enquanto a DCGAN gerou mais imagens aceites, embora com cobertura de apenas 5 classes no conjunto final. Como o aumento de dados ficou pequeno e enviesado, a submissão final usa a baseline original, que teve melhor F1 ponderado no teste interno.

AI Acknowledgement

Usamos IA generativa, como apoio à programação, depuração e organização do relatório. A IA foi usada para sugerir otimizações no código, ajudar a explicar escolhas metodológicas e verificar a consistências entre o notebook e o relatório. Todas as decisões experimentais, execuções, resultados reportados, interpretações e conclusões foram tomadas e validadas por nós.

As execuções do projecto foram realizadas em ambientes com GPU, incluindo Google Colab e Lightning AI, usados para treinar e avaliar os modelos de forma mais rápida [9,10].

Referências

1. Paszke, A., Gross, S., Massa, F., et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS (2019)
2. Kingma, D.P., Welling, M.: Auto-Encoding Variational Bayes. International Conference on Learning Representations (2014)
3. Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al.: Generative Adversarial Nets. NeurIPS (2014)
4. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. NeurIPS (2017)

5. Salimans, T., Goodfellow, I., Zaremba, W., et al.: Improved Techniques for Training GANs. NeurIPS (2016)
6. Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE Transactions on Image Processing (2004)
7. TorchMetrics: Machine learning metrics for PyTorch, <https://torchmetrics.readthedocs.io/>. Acedido em 1 jun. 2026
8. Kaggle: Competition platform and submission format, <https://www.kaggle.com/>. Acedido em 1 jun. 2026
9. Google: Google Colaboratory, cloud-based notebooks with GPU support, <https://colab.research.google.com/>. Acedido em 1 jun. 2026
10. Lightning AI: Lightning AI Studio, cloud development environment with GPU support, <https://lightning.ai/>. Acedido em 1 jun. 2026